



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Department of Computer Science & Engineering (Cyber Security)

Operating System Lab Manual

(22CY44)

Prepared by:

Padmavathi S

Assistant Professor

Dept of CSCY

DSCE

1. Implementation of basic UNIX commands using file API's- Write a program to implement commands ls(-l options), cp, mv and rm using file API's.

ls -l

```
#include<stdio.h>
#include<dirent.h>
#include<sys/stat.h>
#include<pwd.h>
#include<grp.h>
#include<time.h>
int main()
{
    DIR *d;
    struct dirent *de;
    struct stat buf;
    int i,j;
    char P[10]="rwxrwxrwx",AP[10]=" ";
    struct passwd *p;
    struct group *g;
    struct tm *t;
    char time[26];
    d=opendir(".");

    while( (de=readdir(d))!=NULL)
    {
        stat(de->d_name,&buf);
        // File Type
        if(S_ISDIR(buf.st_mode))
            printf("d");
        else if(S_ISREG(buf.st_mode))
            printf("-");
```

```
else if(S_ISCHR(buf.st_mode))
    printf("c");
else if(S_ISBLK(buf.st_mode))
    printf("b");
else if(S_ISLNK(buf.st_mode))
    printf("l");
else if(S_ISFIFO(buf.st_mode))
    printf("p");
else if(S_ISSOCK(buf.st_mode))
    printf("s");
```

//File Permissions P-Full Permissions AP-Actual Permissions

```
for(i=0,j=(1<<8);i<9;i++,j>=>=1)
    AP[i]= (buf.st_mode & j ) ? P[i] : '-';
printf("%s",AP);
```

//No. of Hard Links

```
printf("%5d",buf.st_nlink);
```

//User Name

```
p=getpwuid(buf.st_uid);
printf(" %.8s",p->pw_name);
```

//Group Name

```
g=getgrgid(buf.st_gid);
printf(" %-8.8s",g->gr_name);
```

//File Size

```
printf(" %8lld",buf.st_size);
```

//Date and Time of modification

```

t=localtime(&buf.st_mtime);
strftime(time,sizeof(time),"%b %d %H:%M",t);
printf(" %s",time);
//File Name
printf(" %s\n",de->d_name);
}
}

```

Output:

```

basavarajs-MacBook-Air:~ Paddy$ cc ls.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
drwxr-xr-x  152 Paddy staff    5168 Jul 03 15:21 .
drwxr-xr-x   5 root  admin    170 Jan 13 17:48 ..
-rw-r--r--   1 Paddy staff   2284 Sep 09 11:11 # Tic-Tac-Toe.py
drwxrwxrwx  13 Paddy staff    442 Mar 26 15:43 .anydesk
drwxr-xr-x   3 Paddy staff    102 Jan 09 18:35 .ask
drwxr-xr-x  16 Paddy staff    544 Jan 04 17:06 .atom
drwxr-xr-x   4 Paddy staff    136 May 30 17:03 .aws
-rw-----   1 Paddy staff  12288 Jan 14 17:30 .bash-profile.swp
-rw-r--r--   1 Paddy staff   5215 Jul 03 14:40 .bash_history
-rw-r--r--   1 Paddy staff    409 Feb 03 16:49 .bash_profile
-rw-r--r--   1 Paddy staff     24 Dec 28 14:25 .bash_profile.pysave
drwxr-xr-x  25 Paddy staff    850 Jul 03 15:14 .bash_sessions
drwxr-xr-x   3 Paddy staff    102 May 01 15:52 .cache

```

cp

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#define BUF_SIZE 8192
int main(int argc, char* argv[]) {
int input_fd, output_fd;
size_t ret_in, ret_out;
char buffer[BUF_SIZE];
if(argc != 3){
printf ("Usage: ./a.out file1 file2");
return 1;
}

```

```

}
input_fd = open (argv [1], O_RDONLY);
if (input_fd == -1) {
perror ("open error");
return 1;
}
output_fd = open(argv[2], O_WRONLY | O_CREAT, 0644);
if(output_fd == -1){
perror("open error");
return 1;
}
while((ret_in = read (input_fd, &buffer, BUF_SIZE)) > 0){
ret_out = write (output_fd, &buffer, ret_in);
if(ret_out != ret_in){
perror("write error") ;
return 1;
}
}
close (input_fd);
close (output_fd);
}

```

Output:

```

basavarajs-MacBook-Air:~ Paddy$ cc cp.c
basavarajs-MacBook-Air:~ Paddy$ cat>f1.txt
hi good morning
^C
basavarajs-MacBook-Air:~ Paddy$ ./a.out f1.txt f2.txt
basavarajs-MacBook-Air:~ Paddy$ cat f2.txt
hi good morning

```

mv

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char* argv[]) {
    int input_fd, output_fd;

    /* Are src and dest file name arguments missing */
    if(argc != 3){
        printf ("Usage: ./a.out file1 file2");
        return 1;
    }

    /* Create input file descriptor */
    input_fd = link(argv [1], argv[2]);
    if (input_fd == -1) {
        perror ("link error");
        return 1;
    }

    /* Create output file descriptor */
    output_fd = unlink(argv[1]);
    if(output_fd == -1){
        perror("unlink");
        return 1;
    }
}
```

output:

```
basavarajs-MacBook-Air:~ Paddy$ cc mv.c
basavarajs-MacBook-Air:~ Paddy$ cat>p1.txt
operating system
^C
basavarajs-MacBook-Air:~ Paddy$ ./a.out p1.txt p2.txt
basavarajs-MacBook-Air:~ Paddy$ cat p1.txt
cat: p1.txt: No such file or directory
basavarajs-MacBook-Air:~ Paddy$ cat p2.txt
operating system
```

rm

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char* argv[]) {
    int output_fd = unlink(argv[1]);
    if(output_fd == -1){
        perror("unlink error");
        return 3;
    }
}
```

output:

```
basavarajs-MacBook-Air:~ Paddy$ cc rm.c
basavarajs-MacBook-Air:~ Paddy$ cat>a1.txt
Process scheduling
^C
basavarajs-MacBook-Air:~ Paddy$ ./a.out a1.txt
basavarajs-MacBook-Air:~ Paddy$ cat a1.txt
cat: a1.txt: No such file or directory
```

2. Develop a c program to implement the process system calls (fork(), exec(), wait())

```
#include<stdio.h>
#include<string.h>
#include<sys/types.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/wait.h>
int main(int argc, char *argv[])
{
printf("Main Function: \n");
int retval;
pid_t pid=fork();
if(pid<0){
printf("Error in fork operation\n");
}
if(pid==0)
{
printf("PID for Child process: %d\nPID of its parent process: %d\n",getpid(),getppid());
execl("/bin/ls","ls","-l",(char *)NULL);
}
else
{
printf("PID of parent process: %d\n",getpid());
wait(&retval);
if(WIFEXITED(retval)==1)
{
printf("Child terminated normally\n"); }
else{
printf("Child terminated abnormally\n");
exit(0);
}
```



```
}  
return 0;  
}
```

output:

```
basavarajs-MacBook-Air:~ Paddy$ vi fork.c  
basavarajs-MacBook-Air:~ Paddy$ cc fork.c  
basavarajs-MacBook-Air:~ Paddy$ ./a.out  
Main Function:  
PID of parent process: 1659  
PID for Child process: 1660  
PID of its parent process: 1659  
= ( )  
PID TTY          TIME CMD  
1575 ttys000      0:00.13 -bash  
1659 ttys000      0:00.00 ./a.out  
Child terminated normally  
PID for child process: 1660 PID of its parent process:
```

3. Write a c program to simulate producer-consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1;
int full = 0;
int empty = 2, x = 0;

void producer()
{
    --mutex;
    ++full;
    --empty;
    x++;
    printf("\nProducer produces " "item %d", x);
    ++mutex;
}

void consumer()
{
    --mutex;
    --full;
    ++empty;
    printf("\nConsumer consumes " "item %d", x);
    x--;
    ++mutex;
}

int main()
{
    int n, i;
    printf("\n1. Press 1 for Producer"
           "\n2. Press 2 for Consumer"
           "\n3. Press 3 for Exit");
```

```

#pragma omp critical
for (i = 1; i > 0; i++) {
    printf("\nEnter your choice:");
    scanf("%d", &n);
    switch (n) {
    case 1:
        if ((mutex == 1)
            && (empty != 0)) {
            producer();
        }
        else {
            printf("Buffer is full!");
        }
        break;
    case 2:
        if ((mutex == 1)
            && (full != 0)) {
            consumer();
        }
        else {
            printf("Buffer is empty!");
        }
        break;
    case 3:
        exit(0);
        break;
    }
}
}
output:

```

```
basavarajs-MacBook-Air:~ Paddy$ vi pro1.c
basavarajs-MacBook-Air:~ Paddy$ cc pro1.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
```

1. Press 1 for Producer
2. Press 2 for Consumer
3. Press 3 for Exit

Enter your choice:1

Producer produces item 1

Enter your choice:1

Producer produces item 2

Enter your choice:1

Buffer is full!

Enter your choice:2

Consumer consumes item 2

Enter your choice:2

Consumer consumes item 1

Enter your choice:2

Buffer is empty!

Enter your choice:█

4. Simulate the following CPU scheduling algorithm

a) FCFS b) SJF c) Round Robin

FCFS

```
#include<stdio.h>

int main()
{
    int n,bt[20],wt[20],tat[20],i,j;
    float avwt=0.0,avtat=0.0;
    printf("Enter total number of processes(maximum 20): ");
    scanf("%d",&n);
    printf("\nEnter Process Burst Time:\n");
    for(i=0;i<n;i++)
    {
        printf("P[%d]: ", i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];
    }

    printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time");
    //calculating turnaround time
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];
        avwt+=wt[i];
        avtat+=tat[i];
    }
}
```

```

printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
}
avwt/=n;
avtat/=n;
printf("\n\nAverage Waiting Time:%f",avwt);
printf("\n\nAverage Turnaround Time:%f\n",avtat);
return 0;
}

```

output:

```

basavarajs-MacBook-Air:~ Paddy$ cc fcfs.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
Enter total number of processes(maximum 20): 3
"Enter total number of processes(maximum 20): ";
Enter Process Burst Time:
P[1]: 27
P[2]: 3
P[3]: 3
);i<n;i++)

Process          Burst Time      Waiting Time      Turnaround Time
P[1]             27              0                27
P[2]             3              27               30
P[3]             3              30               33

Average Waiting Time:19.000000
Average Turnaround Time:30.000000

```

SJF

```

#include<stdio.h>

int main()
{
int bt[20],p[20],wt[20],tat[20],i,j,n,total=0,pos,temp;

float avg_wt,avg_tat;

printf("Enter number of process: ");

scanf("%d",&n);

printf("\nEnter Burst Time:\n");

```

```
for(i=0;i<n;i++)
{
printf("p%d: ",i+1);
scanf("%d",&bt[i]);
p[i]=i+1;
}
for(i=0;i<n;i++)
{
pos=i;
for(j=i+1;j<n;j++)
{
if(bt[j]<bt[pos])
pos=j;
}
temp=bt[i];
bt[i]=bt[pos];
bt[pos]=temp;
temp=p[i];
p[i]=p[pos];
p[pos]=temp;
}
wt[0]=0;
for(i=1;i<n;i++)
{
wt[i]=0;
for(j=0;j<i;j++)
wt[i]+=bt[j];
total+=wt[i];
}
avg_wt=(float)total/n;
total=0;
```

```

printf("\nProcess \tBurst Time \tWaiting Time \tTurnaround Time");
for(i=0;i<n;i++)
{
tat[i]=bt[i]+wt[i];
total+=tat[i];

printf("\np%d\t\t %d\t\t%d\t\t%d",p[i],bt[i],wt[i],tat[i]);
}
avg_tat=(float)total/n;
printf("\n\nAverage Waiting Time=%f",avg_wt);
printf("\n\nAverage Turnaround Time=%f\n",avg_tat);
return 0;
}

```

output:

```

basavarajs-MacBook-Air:~ Paddy$ cc sjf.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
Enter number of process: 3

Enter Burst Time:
p1: 12
p2: 3
p3: 5

Process Burst Time Waiting Time Turnaround Time
p2 3 0 3
p3 5 3 8
p1 12 8 20

Average Waiting Time=3.666667
Average Turnaround Time=10.333333

```

RR

```
#include<stdio.h>
```



```

int main()
{

    int cnt,j,n,t,remain,flag=0,tq;
    int wt=0,tat=0,at[10],bt[10],rt[10];
    printf("Enter Total Process:\t ");
    scanf("%d",&n);
    remain=n;
    for(cnt=0;cnt<n;cnt++)
    {
        printf("Enter Arrival Time and Burst Time for Process Process Number %d :",cnt+1);
        scanf("%d",&at[cnt]);
        scanf("%d",&bt[cnt]);
        rt[cnt]=bt[cnt];
    }
    printf("Enter Time Quantum:\t");
    scanf("%d",&tq);
    printf("\n\nProcess\t|Turnaround Time|Waiting Time\n\n");
    for(t=0,cnt=0;remain!=0;)
    {
        if(rt[cnt]<=tq && rt[cnt]>0)
        {
            t+=rt[cnt];
            rt[cnt]=0;
            flag=1;
        }
        else if(rt[cnt]>0)
        {
            rt[cnt]-=tq;
            t+=tq;
        }
    }
}

```

```

if(rt[cnt]==0 && flag==1)
{
    remain--;
    printf("P[%d]\t\t%d\t\t%d\n",cnt+1,t-at[cnt],t-at[cnt]-bt[cnt]);
    wt+=t-at[cnt]-bt[cnt];
    tat+=t-at[cnt];
    flag=0;
}
if(cnt==n-1)
    cnt=0;
else if(at[cnt+1]<=t)
    cnt++;
else
    cnt=0;
}
printf("\nAverage Waiting Time= %f\n",wt*1.0/n);
printf("Avg Turnaround Time = %f",tat*1.0/n);

return 0;
}
output:

```

```

basavarajs-MacBook-Air:~ Paddy$ cc rr.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
Enter Total Process:      3
Enter Arrival Time and Burst Time for Process Process Number 1 :0 2
Enter Arrival Time and Burst Time for Process Process Number 2 :1 6
Enter Arrival Time and Burst Time for Process Process Number 3 :2 1
Enter Time Quantum:      2

Process |Turnaround Time|Waiting Time
P[1]    |      2      |      0
P[3]    |      3      |      2
P[2]    |      8      |      2

Average Waiting Time= 1.333333
Avg Turnaround Time = 4.333333

```

Simulate bankers algorithm for dead lock avoidance

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // P0, P1, P2, P3, P4 are the Process names here
```

```
    int n, m, i, j, k;
```

```
    n = 5; // Number of processes
```

```
    m = 3; // Number of resources
```

```
    int alloc[5][3] = { { 0, 1, 0 }, // P0    // Allocation Matrix
```

```
                        { 2, 0, 0 }, // P1
```

```
                        { 3, 0, 2 }, // P2
```

```
                        { 2, 1, 1 }, // P3
```

```
                        { 0, 0, 2 } }; // P4
```

```

int max[5][3] = { { 7, 5, 3 }, // P0   // MAX Matrix
                  { 3, 2, 2 }, // P1
                  { 9, 0, 2 }, // P2
                  { 2, 2, 2 }, // P3
                  { 4, 3, 3 } }; // P4

```

```

int avail[3] = { 3, 3, 2 }; // Available Resources

```

```

int f[n], ans[n], ind = 0;
for (k = 0; k < n; k++) {
    f[k] = 0;
}
int need[n][m];
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
}
int y = 0;
for (k = 0; k < 5; k++) {
    for (i = 0; i < n; i++) {
        if (f[i] == 0) {

            int flag = 0;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]){
                    flag = 1;
                    break;
                }
            }
        }
    }
}

```

```

        if (flag == 0) {
            ans[ind++] = i;
            for (y = 0; y < m; y++)
                avail[y] += alloc[i][y];
            f[i] = 1;
        }
    }
}

int flag = 1;

for(int i=0;i<n;i++)
{
    if(f[i]==0)
    {
        flag=0;
        printf("The following system is not safe");
        break;
    }
}

if(flag==1)
{
    printf("Following is the SAFE Sequence\n");
    for (i = 0; i < n - 1; i++)
        printf(" P%d ->", ans[i]);
    printf(" P%d", ans[n - 1]);
}

return (0);
}

```

Output:

```
basavarajs-MacBook-Air:~ Paddy$ cc bank.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
Following is the SAFE Sequence
P1 -> P3 -> P4 -> P0 -> P2
```

Simulate all page replacement algorithms

a) FIFO b) LRU c) OPTIMAL

FIFO

// C program for FIFO page replacement algorithm

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int incomingStream[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
```

```
    int pageFaults = 0;
```

```
    int frames = 3;
```

```
    int m, n, s, pages;
```

```
    pages = sizeof(incomingStream)/sizeof(incomingStream[0]);
```

```
    printf("Incoming Frame 1 Frame 2 Frame 3");
```

```
    int temp[frames];
```

```
    for(m = 0; m < frames; m++)
```

```
    {
```

```
        temp[m] = -1;
```

```
    }
```

```

for(m = 0; m < pages; m++)
{
    s = 0;

    for(n = 0; n < frames; n++)
    {
        if(incomingStream[m] == temp[n])
        {
            s++;
            pageFaults--;
        }
    }
    pageFaults++;

    if((pageFaults <= frames) && (s == 0))
    {
        temp[m] = incomingStream[m];
    }
    else if(s == 0)
    {
        temp[(pageFaults - 1) % frames] = incomingStream[m];
    }

    printf("\n");
    printf("%d\t",incomingStream[m]);
    for(n = 0; n < frames; n++)
    {
        if(temp[n] != -1)
            printf(" %d\t", temp[n]);
        else

```

```

        printf(" - \t");
    }
}

printf("\nTotal Page Faults:\t%d\n", pageFaults);
return 0;
}

```

output:

```

basavarajs-MacBook-Air:~ Paddy$ vi fifo.c
basavarajs-MacBook-Air:~ Paddy$ cc fifo.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
Incoming Frame 1 Frame 2 Frame 3
7           7           -           -
0           7           0           -
1           7           0           1
2           2           0           1
0           2           0           1
3           2           3           1
0           2           3           0
4           4           3           0
2           4           2           0
3           4           2           3
0           0           2           3
3           0           2           3
2           0           2           3
1           0           1           3
2           0           1           2
0           0           1           2
1           0           1           2
7           7           1           2
0           7           0           2
1           7           0           1
Total Page Faults:      15

```


LRU

```
#include<stdio.h>
```

```
#include<limits.h>
```

```
int checkHit(int incomingPage, int queue[], int occupied){  
    for(int i = 0; i < occupied; i++){  
        if(incomingPage == queue[i])  
            return 1;  
    }  
    return 0;  
}
```

```
void printFrame(int queue[], int occupied)  
{  
    for(int i = 0; i < occupied; i++)  
        printf("%d\t",queue[i]);  
}
```

```
int main()  
{  
    int incomingStream[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};  
    int n = sizeof(incomingStream)/sizeof(incomingStream[0]);  
    int frames = 3;  
    int queue[n];  
    int distance[n];  
    int occupied = 0;  
    int pagefault = 0;  
  
    printf("Page Frame1 Frame2 Frame3\n");  
  
    for(int i = 0; i < n; i++)
```

```

{
    printf("%d: \t",incomingStream[i]);
    if(checkHit(incomingStream[i], queue, occupied)){
        printFrame(queue, occupied);
    }
    else if(occupied < frames){
        queue[occupied] = incomingStream[i];
        pagefault++;
        occupied++;
        printFrame(queue, occupied);
    }
    else{

        int max = INT_MIN;
        int index;
        for (int j = 0; j < frames; j++)
        {
            distance[j] = 0;
            for(int k = i - 1; k >= 0; k--)
            {
                ++distance[j];

                if(queue[j] == incomingStream[k])
                    break;
            }
            if(distance[j] > max){
                max = distance[j];
                index = j;
            }
        }
        queue[index] = incomingStream[i];
    }
}

```

```

        printFrame(queue, occupied);
        pagefault++;
    }
    printf("\n");
}
printf("Page Fault: %d",pagefault);
return 0;
}

```

output:

```

basavarajs-MacBook-Air:~ Paddy$ cc lru.c
basavarajs-MacBook-Air:~ Paddy$ ./a.out
Page Frame1 Frame2 Frame3
7:      7
0:      7      0
1:      7      0      1
2:      2      0      1
0:      2      0      1
3:      2      0      3
0:      2      0      3
4:      4      0      3
2:      4      0      2
3:      4      3      2
0:      0      3      2
3:      0      3      2
2:      0      3      2
1:      1      3      2
2:      1      3      2
0:      1      0      2
1:      1      0      2
7:      1      0      7
0:      1      0      7
1:      1      0      7

```

OPTIMAL

Write a c program to simulate the following contiguous memory allocation techniques

a) First fit b) best fit c) worst fit

First fit

Worst fit

```
#include<stdio.h>
#define max 25
int main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp;
static int bf[max],ff[max];
printf("\n\tMemory Management Scheme - First Fit");
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{
printf("Block %d:",i);
scanf("%d",&b[i]);
}
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{
```

```

printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
{
temp=b[j]-f[i];
if(temp>=0)
{
ff[i]=j;
break;
}
}
}
frag[i]=temp;
bf[ff[i]]=1;
}
printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
for(i=1;i<=nf;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
return 0;
}

```

Write a c program to simulate disk scheduling algorithms

a)FCFS b) SCAN c)C-SCAN

FCFS

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
#include <math.h>
```

```

int size = 8;
void FCFS(int arr[],int head)
{
    int seek_count = 0;
    int cur_track, distance;

    for(int i=0;i<size;i++)
    {
        cur_track = arr[i];
        distance = abs(head - cur_track);
        seek_count += distance;
        head = cur_track;
    }
    printf("Total number of seek operations: %d\n",seek_count);
    printf("Seek Sequence is\n");

    for (int i = 0; i < size; i++) {
        printf("%d\n",arr[i]);
    }
}

int main()
{
    int arr[8] = { 98,183,37,122,14,124,65,67};
    int head = 50;

    FCFS(arr,head);

    return 0;
}
SCAN
#include<stdio.h>

```

```

#include<stdlib.h>

int main()
{
    int queue[20],n,head,i,j,k,seek=0,max,diff,temp,queue1[20],queue2[20],
        temp1=0,temp2=0;
    float avg;
    printf("Enter the max range of disk\n");
    scanf("%d",&max);
    printf("Enter the initial head position\n");
    scanf("%d",&head);
    printf("Enter the size of queue request\n");
    scanf("%d",&n);
    printf("Enter the queue of disk positions to be read\n");
    for(i=1;i<=n;i++)
    {
        scanf("%d",&temp);
        if(temp>=head)
        {
            queue1[temp1]=temp;
            temp1++;
        }
        else
        {
            queue2[temp2]=temp;
            temp2++;
        }
    }
    for(i=0;i<temp1-1;i++)
    {
        for(j=i+1;j<temp1;j++)
        {

```

```

        if(queue1[i]>queue1[j])
        {
            temp=queue1[i];
            queue1[i]=queue1[j];
            queue1[j]=temp;
        }
    }
}
for(i=0;i<temp2-1;i++)
{
    for(j=i+1;j<temp2;j++)
    {
        if(queue2[i]<queue2[j])
        {
            temp=queue2[i];
            queue2[i]=queue2[j];
            queue2[j]=temp;
        }
    }
}
for(i=1,j=0;j<temp1;i++,j++)
queue[i]=queue1[j];
queue[i]=max;
for(i=temp1+2,j=0;j<temp2;i++,j++)
queue[i]=queue2[j];
queue[i]=0;
queue[0]=head;
for(j=0;j<=n+1;j++)
{
    diff=abs(queue[j+1]-queue[j]);
    seek+=diff;
}

```



```
        printf("Disk head moves from %d to %d with seek\n",queue[j],queue[j+1],diff);
    }
    printf("Total seek time is %d\n",seek);
    avg=seek/(float)n;
    printf("Average seek time is %f\n",avg);
    return 0;
}
```

Program 1

ls -l

```
struct dirent {
    ino_t      d_ino;    // inode number
    off_t      d_off;    // offset to next dirent
    unsigned short d_reclen; // length of this record
    unsigned char d_type; // type of file
    char        d_name[256]; // filename (null-terminated)
};
```

```
struct stat {
    dev_t  st_dev;    // Device ID
    ino_t  st_ino;    // Inode number
    mode_t st_mode;   // File type and permissions
    nlink_t st_nlink; // Number of hard links
    uid_t  st_uid;    // User ID of owner
    gid_t  st_gid;    // Group ID of owner
    dev_t  st_rdev;   // Device ID (if special file)
    off_t  st_size;   // File size in bytes
    blksize_t st_blksize; // Block size for filesystem I/O
    blkcnt_t st_blocks; // Number of blocks allocated
    time_t  st_atime; // Last access time
    time_t  st_mtime; // Last modification time
    time_t  st_ctime; // Last status change time
};
```